CSC1015F Assignment 10

Consolidation

Assignment Instructions

This assignment involves constructing Python programs that use input and output statements, '`if`' and '`if-else`' control flow statements, '`while`' statements, '`for`' statements, and statements that perform `string` manipulation.

NOTE Your solutions to this assignment will be evaluated for correctness and for the following qualities:

- Documentation
  - Use of comments at the top of your code to identify program purpose, author and date.
  - Use of comments within your code to explain each non-obvious functional unit of code.
- General style/readability
  - The use of meaningful names for variables and functions.
- Algorithmic qualities
  - Efficiency, simplicity

These criteria will be manually assessed by a tutor and commented upon. In this assignment, up to 10 marks will be deducted for deficiencies.

Question 1 [35 marks]

Write a module of utility functions called `util.py` for manipulating 2-dimensional arrays of size 4x4. (These functions will be used in Question 3.)

The functions you need to write are as follows:

```
def create_grid(grid):
    """create a 4x4 array of zeroes within grid"""

def print_grid (grid):
    """print out a 4x4 grid in 5-width columns within a box"""

def check_lost (grid):
    """return True if there are no 0 values and there are no
    adjacent values that are equal; otherwise False"""


def check_won (grid):
   """return True if a value>=32 is found in the grid; otherwise
   False"""


def copy_grid (grid):
    """return a copy of the given grid"""

def grid_equal (grid1, grid2):
    """check if 2 grids are equal - return boolean value"""
```

Note: these functions are described using docstrings.

Use the `testutil.py` test program to test your functions. This program takes a single integer input value and runs the corresponding test on your module. This is a variant of unit testing, where test cases are written in the form of a program that tests your program. You will learn more about unit testing in future CS courses.

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*)*:*

2

```
+--------------------+
|2         2         |
|     4          8    |
|     16         128  |
|2    2    2    2    |
+--------------------+
```

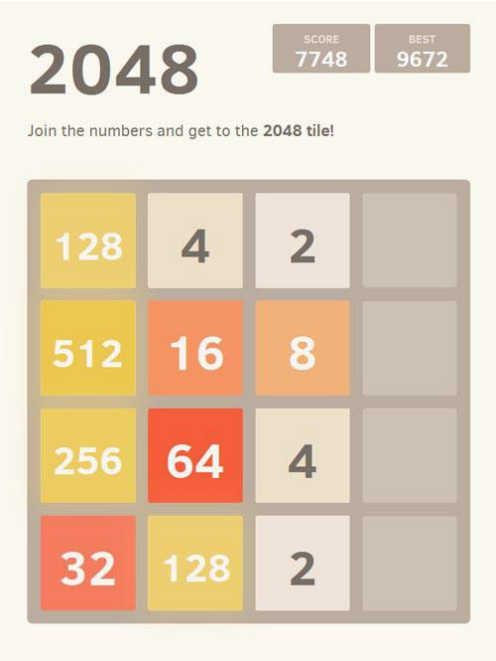
Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this):*
7
`True`

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this):*
17
```
4 4
16 16
2 2
64 4
64 16
64 2
```

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this):*
9
`True`

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this):*
18
`True`

Question 2 [35 marks]



2048 is a puzzle game where the goal is to repeatedly merge adjacent numbers in a grid until the number 2048 is found. Your task in this question is to complete the code for a 2048 program, using the utility module (`util.py`) from Question 2 and a supplied main program (`2048.py`).

The heart of the game is the set of merging functions that merge adjacent equal values and eliminate gaps - you are required ONLY to write these functions in a module named `push.py`:

```
def push_up (grid):
    """merge grid values upwards"""

def push_down (grid):
    """merge grid values downwards"""

def push_left (grid):
    """merge grid values left"""

def push_right (grid):
    """merge grid values right"""
```

The original game can be played at: http://gabrielecirulli.github.io/2048/

Note:

- The `check_won`() function from `util.py` assumes you have won when you reach 32 - this is simply to make testing easier.
- The random number generator has been set to generate the same values each time for testing purposes.

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this):*

```
+--------------------+
|                    |
|                    |
|          2         |
|2                   |
+--------------------+
Enter a direction:
l
+--------------------+
|                    |
|                    |
|2                   |
|2         2         |
+--------------------+
Enter a direction:
u
```

```
+--------------------+
|4          2        |
|                    |
|                    |
|      4             |
+--------------------+
Enter a direction:
d
+--------------------+
|      2             |
|                    |
|                    |
|4     4     2       |
+--------------------+
Enter a direction:
r
+--------------------+
|                2   |
|                    |
|      2             |
|           8    2   |
+--------------------+
Enter a direction:
x
```

Question 3 [30 marks]

Write a Python program called `digit_sum_squares.py` that recursively finds and prints all numbers between two given integers M and N (inclusive) such that the sum of the squares of their digits is a prime number. Each valid number should be printed on a new line.

For example, find all numbers between 10 and 20 such that the sum of squares of their digits is a prime number:

```
10    # 1² + 0² = 1 (not prime)
11    # 1² + 1² = 2 (prime)
12    # 1² + 2² = 5 (prime)
13    # 1² + 3² = 10 (not prime)
14    # 1² + 4² = 17 (prime)
15    # 1² + 5² = 26 (not prime)
16    # 1² + 6² = 37 (prime)
17    # 1² + 7² = 50 (not prime)
18    # 1² + 8² = 65 (not prime)
19    # 1² + 9² = 82 (not prime)
20    # 2² + 0² = 4 (not prime)
```

DO NOT USE any iteration at all (*yes, here we go again!)*

Hint: You must use recursion in the following components:

- Prime checking
- Sum of the squares of digits
- Range traversal from M to N

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter the starting point M:
20
Enter the ending point N:
30
Numbers between 20 and 30 with prime digit square sums:
21
23
25
27
```

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter the starting point M:
4500
Enter the ending point N:
4600
Numbers between 4500 and 4600 with prime digit square sums:
4500
4511
4515
4524
4528
4533
```

```
4539
4542
4544
4551
4566
4577
4582
4593
```

Submission

Create and submit a Zip file called '`ABCXYZ123.zip`' (where ABCXYZ123 is YOUR student number) containing `util.py`, `push.py` and `histogram.py`.

NOTES:

1. FOLDERS ARE NOT ALLOWED IN THE ZIP FILE.
2. As you will submit your assignment to the Automarker, the Assignment tab may say something like "Not Complete". THIS IS COMPLETELY NORMAL. IGNORE IT.